

Serving GLM-5.3 NVFP4 with vLLM

From environment preparation to an observable eight-GPU inference service

A practical engineering account of GLM-5.3-family deployment on NVIDIA Hopper: dependency compatibility, quantised experts, sparse attention, distributed execution, memory budgeting, validation and operational interpretation.

ONE MODEL SERVICE / EIGHT GPU WORKERS



Reference platform: one Linux host, 8 × H100 80 GB, tensor parallelism of eight. **Observed stack:** vLLM 0.29.0, PyTorch 2.13.0+cu130 and FlashInfer 0.6.18.

This guide separates direct observations from engineering recommendations. It generalises the deployment process without identifying a particular model repository or checkpoint revision. The command templates require the operator to select an approved GLM-5.3-family NVFP4 checkpoint.

Prepared in British English • 19 September 2026

What this guide establishes

Observed outcome. A full GLM-5.3-family NVFP4 checkpoint loaded across eight H100 GPUs, completed startup and served a chat-completions request with HTTP 200. Separate single-GPU tests had already exercised quantised matrix multiplication, routed experts, sparse MLA decode and the serving pipeline.

This is a deployment case study and a reproducible starting procedure. It is not a certification of every GLM-5.3 derivative, a quality evaluation or a production capacity benchmark. Modified variants can change quantisation exclusions, tensor names, attention patterns, tokenizer behaviour and speculative-decoding support.

Evidence labels

Label	Meaning
Observed	Reported script output or supplied full-model startup/inference log.
Interpretation	A technical explanation consistent with that evidence; its limits are stated.
Recommended	An operational choice proposed for a fresh deployment, not proof that the exact command was used in the recorded run.

The final launch command and complete environment export were not supplied. Some initial log lines are truncated. Therefore the launch template in this guide is a baseline derived from tested settings, not a byte-for-byte reconstruction of the successful invocation.

Reading route

Pages	Topic
3-6	Model structure, vLLM responsibilities, host resources and compatibility.
7-10	Installation, persistent paths, acquisition and metadata checks.
11-16	Validation, quantisation, sparse attention and distributed execution.
17-18	Full-model launch and curl inference.
19-23	Startup phases, memory, graph execution and request metrics.
24-27	Controlled tuning, operations, troubleshooting and handover.
28	Sources and evidence provenance.

Commands assume Bash, Ubuntu 24.04, Python 3.12 and administrative access inside a prepared GPU environment. `/workspaces` is an example mounted data root, independent of any hosting provider. GPU commands are intended for the deployment host; they were not executed by the author during preparation of this document.

Four independent design choices

The model family combines a large mixture-of-experts network, multi-head latent attention, sparse token selection and checkpoint quantisation. Treat these as separate compatibility dimensions.

Mixture of experts (MoE). A router selects a small subset of expert feed-forward networks for each token. The exercised configuration had 256 routed experts and selected eight per token, alongside a shared expert. Sparse activation reduces the work per token; it does not eliminate the need to store the expert weights available to routing.

Multi-head latent attention (MLA). Attention retains a compressed latent representation rather than a conventional full per-head K/V representation. The exercised geometry used a 512-element latent and a 64-element rotary-position component, giving 576 cache elements in the tested decode layout. Query projection dimensions and cache dimensions are not interchangeable.

Sparse attention (DSA). An indexer identifies relevant cached positions. The selected attention kernel then computes attention over those positions. The observed configuration used a sparse top-k of 2,048. This is a selection budget, not the maximum sequence length. A sparse decode test with predetermined indices validates the consumer kernel, not the indexer that chooses those indices.

NVFP4 checkpoint representation. Routed expert weights were encoded in a ModelOpt NVFP4 format with associated scale tensors. Other components remained at higher precision. A four-bit model label therefore does not mean that all weights, activations, intermediate buffers and KV entries occupy four bits.

Exercised property	Engineering consequence
GLMoeDsaForCausalLM	The runtime must recognise the architecture and its tensor mapping.
Hidden size 6,144; 64 attention heads	Partition geometry influences kernel selection and local matrix shapes.
78 main layers; one MTP layer	The optional prediction head introduces a further execution path.
Full/shared attention pattern	Cache and index reuse must follow the implementation, not a generic dense-model formula.
NVFP4 experts with BF16 components	Several kernel families and precision rules coexist in one forward pass.

These values describe the configuration examined in the case study. For another variant, read its configuration and safetensors index before reusing dimension-dependent tests or capacity estimates.

What vLLM actually does

vLLM turns a checkpoint into a continuously scheduled inference service. Loading weights is only one part of that work. It resolves model metadata, chooses compatible GPU implementations, allocates state, coordinates distributed computation and converts generated tokens into API responses.

Runtime component	Responsibility in this deployment
API server	Accepts HTTP, validates requests, applies input processing and returns or streams results.
Engine core	Schedules requests, manages logical KV-cache allocation and coordinates execution.
GPU workers	Own device-local model state and execute their partitions of the forward pass.
Model runner	Prepares execution inputs and attention metadata; invokes kernels and graph execution paths.
Backend implementations	Provide attention, quantised GEMM, fused MoE, normalisation and communication operations.

For a conventional single-node TP=8, PP=1, DP=1 process arrangement, there is one API process, one engine core and eight GPU workers, in addition to support processes or threads. This division explains the different process labels in the log. The process architecture is documented by vLLM [1]; the supplied log directly confirms the engine and eight ranks.

During a request

The input template and tokenizer produce token IDs. The scheduler admits work and allocates cache blocks. Prefill evaluates prompt tokens and writes reusable attention state. Decode then advances generation, usually one token per active sequence per ordinary step. The workers execute attention and expert computation, exchange distributed results where necessary, and produce logits for sampling.

KV cache avoids recomputing all prior attention state on each step. Its block-based allocation also lets requests enter and leave the active batch as they progress. Completed requests release their active allocation for reuse. Optional prefix caching can reuse matching prompt state; it does not cache an arbitrary semantic answer.

Output processing applies stopping rules, detokenises tokens and constructs response fields. Reasoning and tool-call parsers interpret the model's output conventions. The `glm45` and `glm47` names identify parser formats; they do not imply that the loaded weights belong to those older model generations.

The service therefore has several distinct readiness levels: importable packages, usable CUDA, viable kernels, loaded distributed weights, warmed execution, a listening API and successful generation. Each requires its own evidence.

Prepare the machine before Python

Recommended baseline: one host with eight visible H100 80 GB GPUs, adequate CPU allocation, sufficient shared memory and a fast local or deliberately provisioned network filesystem. Prefer a well-connected NVLink/NVSwitch topology for frequent tensor-parallel communication; eight devices listed by the driver alone do not establish interconnect quality.

```
nvidia-smi
nvidia-smi topo -m
df -h /workspaces /dev/shm
findmnt -T /workspaces
free -h
lscpu
ulimit -n
```

Check GPU count, model, compute mode, existing memory use and topology. Investigate PCIe-only bottlenecks and cross-socket paths before diagnosing the framework. Device exposure and NVIDIA container runtime configuration belong to the host or orchestrator, not to pip.

Storage. The observed checkpoint comprised 282 shards totalling about 464.82 GB, or 432.90 GiB. Allow additional room for the Python environment, build artefacts, downloads, logs and caches. A practical starting allocation is 800 GB to 1 TB of free storage for this size of checkpoint; it is headroom, not a family-wide requirement. Avoid accidentally keeping a second complete copy.

Host RAM. The successful log reported about 1,411.87 GiB available. That demonstrates a generous host configuration, not a minimum RAM requirement. Loading strategy, page cache, tensor conversion and worker concurrency determine the actual peak. Approximately 1 TB is a conservative provisioning target for a checkpoint of this size, subject to measurement. Monitor the container's memory limit as well as host-wide free memory.

Shared memory and CPU. A default 64 MB /dev/shm is a poor starting point for multi-process GPU serving. Provision several GiB or more according to workload and organisational policy. CPU starvation can delay tokenisation, scheduling, loading and JIT even when the GPUs are healthy. Leave resources for all workers and the API process.

Creating /workspaces does not mount persistent storage. Confirm its filesystem and lifecycle explicitly. Keep weights, caches and logs on the intended mount. The observed filesystem was classified as OVERLAY; that classification alone does not identify its underlying disks or guarantee persistence.

Driver, runtime, toolkit and ABI

Layer	Observed value or role
Host NVIDIA driver	580.126.09; provides the host-side GPU interface.
GPU architecture	H100, compute capability 9.0 (Hopper / SM90).
Python	3.12 in a dedicated virtual environment.
vLLM	0.29.0; Python control code and native GPU extensions.
PyTorch	2.13.0+cu130; CUDA runtime reported as 13.0.
FlashInfer	0.6.18; kernels, JIT facilities and autotuning support.
Base user-space environment	Ubuntu 24.04; initially included an older PyTorch/CUDA stack.

The CUDA version shown by `nvidia-smi` describes driver capability, not the version of every toolkit installed in the container. `torch.version.cuda` identifies the CUDA version associated with the PyTorch build. `nvcc --version` identifies the local compiler, if present. These are three different observations.

CUDA 13.x requires a compatible driver branch; NVIDIA lists a minimum driver family of 580 for minor-version compatibility [2]. The observed driver met that condition. Do not attempt to replace the host driver by installing a package inside the application container.

Prebuilt wheels can supply user-space CUDA dependencies independently of an older base-image toolkit. However, compilation paths may still invoke local tools. The successful run warned that newer NVCC would improve a DeepGEMM path. A functional runtime is therefore not proof that every local compiler path is optimal.

Use an isolated virtual environment without inherited system packages. Let the dependency resolver maintain the vLLM/PyTorch/native-extension combination. Avoid changing PyTorch underneath an existing vLLM wheel or forcing an incompatible installation with `--no-deps`. vLLM's installation guidance describes this binary-compatibility concern [3].

The earlier `ModuleNotFoundError: vllm._C` was an invalid diagnostic for the installed layout: native libraries used names including `_C_stable_libtorch`. Inspect package location and distribution files, then exercise supported operations. A successful import of `_custom_ops` is a useful initial check, but actual GPU execution is stronger evidence.

Build an isolated serving environment

The following is a version-scoped reconstruction of the tested core stack. Execute it in Bash on the target machine. Retain the resolver output. The exact Transformers version from the original test environment was not recorded; the range below is an installation constraint, not a complete lock file.

```
apt-get update
apt-get install -y --no-install-recommends \
  ca-certificates curl git build-essential \
  python3-venv python3-dev ninja-build tmux

mkdir -p /workspaces/{venvs,models,hf,cache,tmp,logs,scripts,secrets}
chmod 700 /workspaces/secrets

python3 -m venv /workspaces/venvs/glm53
source /workspaces/venvs/glm53/bin/activate
python -m pip install --upgrade pip uv

uv pip install \
  'vllm==0.29.0' \
  'torch==2.13.0+cu130' \
  'flashinfer-python==0.6.18' \
  'transformers>=5.15.0,<6' \
  --torch-backend=cu130

uv pip install 'flashinfer-cubin==0.6.18' \
  --extra-index-url https://flashinfer.ai/whl
uv pip install 'flashinfer-jit-cache==0.6.18' \
  --extra-index-url https://flashinfer.ai/whl/cu130

python -m pip check
python -m pip freeze > /workspaces/requirements-installed.txt
flashinfer show-config
```

FlashInfer provides a Python package, precompiled cubins and a CUDA-specific JIT-cache package [4]. Matching auxiliary packages can reduce startup downloads and compilation. They do not guarantee that every shape or execution path is already warmed, nor that vLLM selects FlashInfer for attention.

Resolve package conflicts before continuing. Save the resolved package list and build a reproducible image or wheel archive from the working environment. A freeze file records versions but does not by itself preserve wheel origins, system libraries, compiler versions or downloaded kernel artefacts.

A CUDA development toolkit is only needed for paths that actually compile with it. If such a path fails, inspect the first compiler error and `nvcc --version`; install a compatible toolkit through the approved image build process. Do not alter several low-level dependencies during the first successful deployment.

Keep paths and identities explicit

The repository used for acquisition, the directory used for loading and the model name exposed by the API serve different purposes. Keep them separate so clients do not depend on host paths.

```
cat > /workspaces/scripts/glm53-env.sh <<'EOF'
export HF_HOME=/workspaces/hf
export XDG_CACHE_HOME=/workspaces/cache
export UV_CACHE_DIR=/workspaces/cache/uv
export PIP_CACHE_DIR=/workspaces/cache/pip
export TRITON_CACHE_DIR=/workspaces/cache/triton
export TORCHINDUCTOR_CACHE_DIR=/workspaces/cache/torchinductor
export TMPDIR=/workspaces/tmp
export PYTHONUNBUFFERED=1
export MODEL_DIR=/workspaces/models/glm53-nvfp4
export SERVED_MODEL_NAME=glm-5.3-nvfp4
EOF

source /workspaces/scripts/glm53-env.sh
source /workspaces/venvs/glm53/bin/activate
```

Weights are the primary durable artefact. **Kernel caches** reduce repeated compilation but are tied to software, architecture and sometimes shapes. **Logs** preserve the resolved execution choices. **Temporary storage** must support concurrent workers and should not silently exhaust the root filesystem.

Treat a virtual environment as rebuildable. Reusing one on a different operating-system image, CPU architecture or library stack is not a reliable migration strategy. Recreate it from the captured dependency set instead.

Create a local API credential

```
python - <<'PY'
import os, secrets
from pathlib import Path
p = Path('/workspaces/secrets/vllm-api-key')
if not p.exists():
    fd = os.open(p, os.O_WRONLY | os.O_CREAT | os.O_EXCL, 0o600)
    with os.fdopen(fd, 'w') as f:
        f.write(secrets.token_urlsafe(32) + '\n')
p.chmod(0o600)
PY
```

Keep credentials out of shared logs and support bundles. An API key is one control; network reachability, TLS termination and access to administrative or monitoring routes must be handled at the service boundary.

Download before allocating the service

Separate network acquisition from model execution. This prevents a download failure from being confused with GPU initialisation and avoids paying for all accelerators while waiting for hundreds of gigabytes to arrive. The same local checkpoint can then support repeated launches.

Set `MODEL_REPO` to the approved GLM-5.3-family NVFP4 repository. Set `MODEL_REF` to the immutable identifier selected by your change process. No particular repository or identifier is prescribed here. Omitting this discipline allows a moving branch to change between deployments.

```
# Optional: required only for a gated/private repository.
hf auth login

read -r -p 'Approved HF repository: ' MODEL_REPO
read -r -p 'Approved immutable model reference: ' MODEL_REF
test -n "$MODEL_REPO" && test -n "$MODEL_REF"

hf download "$MODEL_REPO" \
  --revision "$MODEL_REF" \
  --local-dir "$MODEL_DIR"
```

Run long downloads in a terminal multiplexer or a supervised job. Preserve the command's exit status. Download the complete snapshot: configuration, tokenizer files, chat template, generation configuration, safetensors index and all referenced shards. The HF CLI supports a separate local-directory download workflow [5].

```
python - <<'PY'
import json, os
from pathlib import Path
p = Path(os.environ['MODEL_DIR'])
assert (p / 'config.json').is_file()
idx = json.loads((p / 'model.safetensors.index.json').read_text())
shards = sorted(set(idx['weight_map'].values()))
missing = [n for n in shards if not (p / n).is_file()]
assert not missing, missing
total = sum((p / n).stat().st_size for n in shards)
print('Shards:', len(shards))
print('GB:', round(total / 1e9, 2))
print('GiB:', round(total / 2**30, 2))
print('PASS: referenced files are present')
PY
```

This verifies presence, not numerical content or full integrity. For a transport or archival workflow, retain a trusted manifest and verify hashes after copying. Generating hashes reads the entire checkpoint and has a real I/O cost. Neither this check nor metadata parsing allocates the full model on the GPU.

Verify the model before loading weights

Architecture registration establishes that vLLM knows a model class. It does not establish that every tensor, quantisation exclusion or runtime backend is compatible. Check the configuration and index together.

```
python - <<'PY'
import json, os
from pathlib import Path
from transformers import AutoConfig, AutoTokenizer
p = Path(os.environ['MODEL_DIR'])
cfg = AutoConfig.from_pretrained(p, local_files_only=True)
print('Config class:', type(cfg).__name__)
print('Architectures:', cfg.architectures)
print('Model type:', cfg.model_type)
raw = json.loads((p / 'config.json').read_text())
print('Quantisation:', raw.get('quantization_config'))
tok = AutoTokenizer.from_pretrained(p, local_files_only=True)
messages = [{'role': 'user', 'content': 'What is 17 * 19?'}]
rendered = tok.apply_chat_template(
    messages, tokenize=False, add_generation_prompt=True)
encoded = tok.apply_chat_template(
    messages, tokenize=True, add_generation_prompt=True,
    return_dict=True)
ids = encoded['input_ids']
assert '17 * 19' in rendered
print('Token count:', len(ids))
print('Rendered suffix:', repr(rendered[-250:]))
print('Decoded suffix:', repr(tok.decode(ids)[-250:]))
PY
```

Observed correction. An early test reported two tokens because it counted the keys of a BatchEncoding (input_ids and attention_mask). Counting the actual ID sequence gave 20 tokens. The rendered and decoded suffix preserved the question and opened the assistant reasoning segment. A container's length is not necessarily the token count.

The checkpoint metadata identified GLMoeDsaConfig, GLMoeDsaForCausalLM, ModelOpt and NVFP4. The index included per-expert scale tensors and 791 tensors under the expected MTP-layer prefix. These findings reduced the risk of a wrong architecture or missing prediction head, but did not execute either component.

Do not enable remote code merely because an unrelated model recipe includes the flag. Use the built-in architecture when supported. If a selected derivative genuinely requires repository code, review that code and its provenance before granting execution permission.

A ladder of evidence, not a single PASS

The inexpensive single-GPU phase isolated likely failure classes before committing to a full eight-GPU run. The aim was to test the actual operations needed by the target, not simply import large libraries.

Check	What passed	Remaining limitation
CUDA and framework	GPU execution, torch.compile / Inductor / Triton; CUDA graph capture and replay.	Does not exercise model-specific kernels or distributed links.
Generic FlashInfer attention	BF16 KV maximum error 0.000444; FP8 KV 0.000530.	A different attention path from GLM sparse MLA.
Model metadata	Architecture, tokenizer, scale tensors and MTP tensors found.	No full weight loading or forward pass.
NVFP4 linear kernels	Repacking and scale preparation; GEMM at two expert matrix shapes.	No router or multi-expert merge.
Routed fused MoE	Synthetic routing, gated activation and weighted expert output.	No full layer stack or distributed collectives.
Sparse MLA decode	BF16 cache; numerical checks with 8 and 64 query heads.	Predetermined sparse indices; not the complete indexer.
Tiny-model API run	Long prefill and 16 decode tokens; eager and graph-enabled runs.	Small BF16 model; no target-sized NVFP4 checkpoint.
Full eight-GPU run	Weights loaded, warm-up completed, chat request returned 200.	No supplied quality suite, concurrency benchmark or MTP run.

Sequence matters. An import failure should be resolved before debugging kernels. A quantisation error should be isolated before loading all layers. A kernel that works in isolation still needs integration testing. Only the full run exposes target-scale loading peaks and real distributed execution.

Keep test inputs, random seeds, reference equations, precision and tolerances with the results. A PASS without those details cannot be repeated reliably. Compare against an independently calculated reference where possible; checking only for finite output is weaker evidence.

The numerical values in this guide are observations from supplied test output. They are not universal tolerances. Expected rounding error depends on the operation, accumulation path, dimensions, scales and activation dtype.

Why Marlin needed its own tests

H100 does not provide the native FP4 execution path of newer hardware. In the exercised implementation, Marlin made NVFP4 expert weights usable through a weight-only quantised path with higher-precision activations. This makes format conversion and scaling part of correctness, not merely a loading optimisation.

The inspected preparation function repacked packed four-bit weights into the layout required by Marlin, padded dimensions as needed, permuted group scales and transformed the global scale. NVFP4 used groups of 16 in this path. The apply function selected the FP4 scalar type and invoked Marlin GEMM.

The test constructed synthetic quantised values and scales, then compared the kernel result with a reference based on the same dequantised weights. It tested execution correctness for that representation; it did not measure quantisation loss against the original unquantised model.

Local expert matrix	Token rows M	Observed relative L2
K=6,144; N=512	1 and 16	0.001639 and 0.001623
K=256; N=6,144	1 and 16	0.001638 and 0.001656

These shapes reflect a hidden size of 6,144 and a partitioned intermediate size of 256. A fused gate/up projection doubles that local intermediate output to 512; the down projection maps 256 back to 6,144. Testing both directions catches different packing and layout errors. Testing M=1 and M=16 exercises decode-like and small batched work.

From GEMM to routed experts

The next synthetic test included multiple experts, selected routes, router weights, gated SiLU computation and the final weighted combination. Relative L2 was 0.004665 for one token and 0.004987 for 16 tokens. This covered the fused expert operation more faithfully than a standalone linear layer.

In these tests, relative L2 means $\| \text{output} - \text{reference} \|_2 / \| \text{reference} \|_2$ for a non-zero reference. Maximum absolute error complements it by exposing the largest individual deviation.

The inspected NVFP4 linear path explicitly rejected one-byte activation modes. That restriction concerns activations entering the quantised GEMM. It is distinct from the dtype of the attention KV cache. Similarly, an advisory about experimental atomic-add reduction is an optional performance experiment, not a requirement for successful execution.

The decisive BF16 KV-cache finding

Observed failure. With effective KV dtype `fp8_e4m3`, the H100 sparse-MLA selector found no valid backend. `FLASH_ATTN_MLA_SPARSE` explicitly rejected FP8 KV; `FLASHMLA_SPARSE` also rejected that cache dtype. Other candidates failed the sparse-attention or device-capability requirements.

This was not contradicted by the successful generic FlashInfer FP8 test. A library may contain a working FP8 attention kernel while the framework's model-specific sparse MLA path cannot use that combination of device, layout and dtype.

An initial selector test passed the requested string `auto` instead of the resolved cache dtype. The model configuration had converted `auto` to FP8. Testing the unresolved value therefore produced a misleading result. Inspect the effective configuration after resolution, then invoke the selector with those exact values.

```
--dtype bfloat16 --kv-cache-dtype bfloat16
```

The first flag controls the general model computation dtype; the second explicitly controls cache storage. For the exercised stack, BF16 KV is the justified baseline. The vLLM cache configuration supports an explicit dtype setting [6]. This finding is scoped to the observed implementation and should be rechecked when upgrading.

Numerical sparse-decode evidence

After the effective cache setting was confirmed as BF16, the selector chose `FLASH_ATTN_MLA_SPARSE`. A test invoked the selected implementation's FlashAttention version 3 entry point using split latent and rotary queries, a latent dimension of 512, a rotary dimension of 64, sparse top-k 2,048 and attention scale 0.0625.

With eight heads, relative L2 was 0.002182 and maximum absolute error 0.000072. With 64 heads, the corresponding values were 0.002134 and 0.000071. Eight heads reflect the target local query-head count under TP=8; 64 exercise the unpartitioned head count.

This proves that the selected BF16 sparse decode operation executed and agreed closely with its reference on synthetic inputs. It does not validate the learned indexer, all cache metadata paths, every sequence length or prefill. The later integration and full-model runs supply additional coverage.

Installing FlashInfer 0.6.18 did not itself select a FlashInfer sparse backend on this host. The actual selector output is the authoritative evidence for the running system.

What a tiny related model can prove

A tiny random model from the GLM-5.2 line was used because it shared the `GLMoeDsaForCausalLM` architecture. It fitted on one H100 and exercised the serving integration without acquiring the full checkpoint. The model-generation label alone did not establish equivalence; shared implementation and compatible configuration were the reason to use it.

The API test submitted 2,333 prompt tokens and requested 16 generated tokens. Both the eager and graph-enabled runs returned HTTP 200 with 2,349 total tokens. The graph-enabled startup explicitly reported successful CUDA graph capture.

This tested HTTP handling, request scheduling, a real model forward path, cache operation, prefill and repeated decode. A prompt longer than the sparse top-k budget was useful for exercising a less trivial attention workload. It still did not establish coverage of every indexing branch or target-scale sequence.

The tiny checkpoint was randomly initialised. Repetitive outputs such as a repeated word or number were consequently not evidence of a deployment fault. They were also not evidence of language quality. Different strings between eager and graph-enabled runs do not prove numerical equivalence; that would require controlled inputs, sampling and output comparisons at a suitable numerical level.

Interpret the small-model logs in their own scope

One tiny-model run reported 14.65 GiB of available KV memory, capacity for 3,318,592 cached tokens and a theoretical concurrency of 810.20 at 4,096 tokens per request. Those numbers came from a small model using a GPU utilisation setting of 0.2. They cannot be transferred to the full 78-layer target.

The reported generation throughput of about 1.6 tokens/s was averaged over a logging interval containing very little work. It was not a meaningful H100 throughput benchmark. Likewise, a prefix-cache hit rate on a repeated test reflected prompt reuse in that workload, not a universal cache benefit.

A warning about Triton JIT compilation of `fused_moe_kernel` during inference meant that a previously unwarmed shape compiled on demand. The request still succeeded. This signals a latency risk for first-use workloads and motivates representative warm-up, rather than an immediate package reinstall.

The tiny model supplied an integration bridge between isolated kernels and the full deployment. It could not replace tests of NVFP4 weight loading, full-model memory, eight-GPU communication or the modified GLM-5.3 checkpoint itself.

Check visibility and actual collectives

The final host must pass a basic device check even when a one-GPU machine already passed. Driver exposure, interconnects and process launch are properties of the target host.

```
python - <<'PY'
from importlib.metadata import version
import torch, flashinfer
from vllm import _custom_ops
for p in ('vllm', 'torch', 'transformers', 'flashinfer-python'):
    print(p, version(p))
assert torch.cuda.is_available()
assert torch.cuda.device_count() == 8
for i in range(8):
    with torch.cuda.device(i):
        x = torch.ones(16, device=f'cuda:{i}')
        assert x.sum().item() == 16
        torch.cuda.synchronize()
    print(i, torch.cuda.get_device_name(i))
print('PASS: imports and all eight devices')
PY
```

For a new or untrusted host topology, this small collective check is recommended. It was not part of the supplied single-GPU evidence. It proves basic NCCL communication, not bandwidth or the custom all-reduce path selected later by vLLM.

```
cat > /workspaces/scripts/check-nccl.py <<'PY'
import os
from datetime import timedelta
import torch
import torch.distributed as dist
rank = int(os.environ['LOCAL_RANK'])
torch.cuda.set_device(rank)
dist.init_process_group('nccl', timeout=timedelta(seconds=120))
x = torch.tensor([float(dist.get_rank() + 1)], device='cuda')
dist.all_reduce(x)
torch.cuda.synchronize()
assert x.item() == 36.0, (rank, x.item())
if dist.get_rank() == 0:
    print('PASS: eight-rank NCCL sum')
dist.destroy_process_group()
PY
torchrn --standalone --nproc-per-node=8 \
/workspaces/scripts/check-nccl.py
```

Do not run this alongside a busy production engine. If it hangs or fails, inspect topology, shared memory, process permissions and the first worker error before loading the checkpoint. A collective failure often causes other ranks to wait; their later timeout is not necessarily the original cause.

Why eight GPUs form one service

Tensor parallelism (TP) partitions suitable model tensors and computation across ranks. A request is processed cooperatively by those ranks. It does not create eight independent model replicas. Communication is part of the forward pass, so topology and collective implementations affect latency.

Pipeline parallelism (PP) distributes layer groups across stages. **Data parallelism (DP)** creates independent serving replicas or replica groups. **Expert parallelism (EP)** distributes expert work and requires appropriate routing/communication. **Context parallelism** partitions attention-related sequence work in supported configurations. These mechanisms solve different scaling problems.

The successful log recorded TP=8, PP=1, DP=1 and decode-context-parallel size 1. Eight workers initialised ranks 0 through 7 using NCCL 2.29.7. The runtime used the V2 Model Runner. These observations describe the actual execution arrangement more reliably than the machine's GPU count alone.

Observed log detail	Interpretation
world_size=8, local_world_size=8	Eight ranks participate on the local host.
TP dispatch: FLASHINFER, CUSTOM, SYMM_MEM, PYNCCCL	Multiple eligible all-reduce implementations are available in priority order. This does not mean all run for every collective.
An ep:0 communicator exists	Internal group construction is visible; this alone does not prove expert-parallel execution was enabled.
MoEPrepareAndFinalizeNoDPEPModular	The observed MoE preparation path did not indicate active DP/EP dispatch.

The baseline deliberately uses TP=8 without enabling EP separately. It establishes a simpler reference before changing expert placement. Do not copy an EP setting from a different checkpoint or GPU generation without measuring its implications for kernels, routing and memory.

The 64 model attention heads corresponded to eight query heads per TP rank in the dimension-specific tests. That arithmetic does not imply that every parameter or cache tensor divides evenly by eight. Some components can be replicated, padded, sharded differently or shared across layers.

When changing TP or EP, revisit shape constraints, local expert dimensions, communication, attention selection and memory on every rank. A single rank reaching its memory limit can stop the entire service even if the aggregate free memory appears sufficient.

Serve the local checkpoint

This command starts the full model on eight GPUs. It is not a metadata test and must not be used to load the target checkpoint on the one-GPU validation machine. Create the script after acquisition and acceptance checks.

```
cat > /workspaces/scripts/serve-glm53.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
source /workspaces/scripts/glm53-env.sh
source /workspaces/venvs/glm53/bin/activate
export VLLM_API_KEY="$(cat /workspaces/secrets/vllm-api-key)"
export HF_HUB_OFFLINE=1
export TRANSFORMERS_OFFLINE=1

vllm serve "$MODEL_DIR" \
  --served-model-name "$SERVED_MODEL_NAME" \
  --tensor-parallel-size 8 \
  --distributed-executor-backend mp \
  --moe-backend marlin \
  --dtype bfloat16 \
  --kv-cache-dtype bfloat16 \
  --max-model-len 32768 \
  --max-num-seqs 2 \
  --max-num-batched-tokens 4096 \
  --enable-chunked-prefill \
  --gpu-memory-utilization 0.90 \
  --tool-call-parser glm47 \
  --reasoning-parser glm45 \
  --enable-auto-tool-choice \
  --host 127.0.0.1 --port 8000 \
  2>&1 | tee -a /workspaces/logs/glm53-vllm.log
EOF
chmod +x /workspaces/scripts/serve-glm53.sh
bash /workspaces/scripts/serve-glm53.sh
```

Recommended settings. Explicit BF16 KV addresses the observed sparse-backend incompatibility. Marlin selects the validated NVFP4 expert path. A 32,768-token context, two concurrent sequences and a 4,096-token scheduling budget provide a conservative initial workload. They are not performance maxima.

The template leaves attention selection and compilation/graph defaults to the installed runtime. It does not enable MTP or EP. The successful log later showed compilation mode NONE with active CUDA graphs; the truncated launch record does not establish which setting produced that combination.

Binding to loopback is deliberate for local verification. For remote access, place the service behind a controlled TLS ingress and change the binding only as required. The offline variables prevent normal HF model retrieval; they are not a network sandbox and do not block every kernel-artifact download.

Verify the API and inspect the response

Wait for model loading, warm-up and application startup to finish. In a second shell, load the same API name and credential. The OpenAI-compatible endpoints and model-name mapping are described in vLLM's server documentation [7].

```
source /workspaces/scripts/glm53-env.sh
export VLLM_API_KEY="$(cat /workspaces/secrets/vllm-api-key)"

curl --fail-with-body -sS \
  http://127.0.0.1:8000/v1/models \
  -H "Authorization: Bearer $VLLM_API_KEY"

curl --fail-with-body -sS \
  http://127.0.0.1:8000/v1/chat/completions \
  -H "Authorization: Bearer $VLLM_API_KEY" \
  -H 'Content-Type: application/json' \
  --data-binary @- \
  -o /workspaces/logs/first-response.json <<JSON
{
  "model": "$SERVED_MODEL_NAME",
  "messages": [
    {"role": "user", "content": "What is 17 * 19? Answer briefly."}
  ],
  "temperature": 0.2,
  "max_tokens": 4096,
  "stream": false
}
JSON

python3 -m json.tool /workspaces/logs/first-response.json
```

The API name is `glm-5.3-nvfp4`, matching `--served-model-name`; the client does not submit the local filesystem path. For this smoke test, the expected arithmetic answer is 323.

Inspect the answer, usage fields and `finish_reason`. HTTP 200 establishes successful request handling, not semantic correctness. A length finish reason means the output budget was exhausted; reasoning may consume the budget before a final answer appears. Increase the budget only within the configured combined prompt/output limit.

Reasoning and tool-call parsing are additional acceptance dimensions. A plain arithmetic request does not prove that structured tool calls parse correctly. Test those fields with representative requests before applications depend on them. Enabling automatic tool choice does not execute tools; the client or an orchestration service must validate and perform any requested action.

For latency measurements, use streaming and record arrival of the first generated token. Curl's time to first response byte can include headers or protocol framing and is not automatically the model's time to first token.

Loading is a sequence of commitments

The full-model log exposed the following progression. Times are local log timestamps from the observed run; they are illustrative, not startup guarantees.

Stage	Observed signal	What it establishes
Configuration, about 11:02:50	Resolved limits, parser settings and graph configuration.	The runtime has assembled an execution configuration; weights are not yet ready.
Workers, 11:03:03-23	Eight ranks, NCCL and communicator dispatch.	Distributed execution machinery is initialised.
Model construction, 11:03:24	V2 runner; sparse attention and Marlin selections.	Compatible implementations have been chosen for this configuration.
Checkpoint traversal, about 11:03:25-11:04:07	282 safetensors shards processed.	The loader progressed through the checkpoint; later preparation still matters.
Profiling and warm-up	Cache budgeting, graph captures and autotune.	Execution state and capacity are being prepared.
API readiness, 11:05:10	Application startup complete; model listing returns 200.	The server is ready to accept requests.
Inference	Active generation and chat-completions 200.	The service has completed an actual request.

During loading, file tensors must be mapped to model parameters and the appropriate rank's slices. Quantised weights may require repacking and scale transformations before their kernels can consume them. Loading a file successfully is therefore not the same as preparing every operator for execution.

The log reported a 432.90 GiB checkpoint, available host RAM of 1,411.87 GiB and an OVERLAY filesystem. Automatic prefetching was disabled because this filesystem was not recognised as NFS or Lustre. This was a loader policy decision, not a missing-weight error.

Do not divide total checkpoint size by the progress-bar duration and publish the result as physical disk bandwidth. Memory mapping, page-cache hits, rank-specific access and deferred page faults can decouple progress from storage reads. Measure block-device or filesystem throughput and cold/warm starts separately.

For slow startup, locate the stalled stage first. Downloading, storage traversal, weight conversion, collectives, JIT, autotuning and graph capture require different remedies. Preserve the first error and timestamps from all ranks; a later engine timeout may only report the consequence.

Several kernel systems coexist

The successful run selected FLASH_ATTN_MLA_SPARSE for sparse attention, FLASH_ATTN for MLA prefill and MARLIN for NVFP4 MoE. These are separate decisions. There is no requirement for one library to implement every operation in a forward pass.

Log signal	Meaning and response
Vendored DeepGEMM fallback	A standalone package was not found, then the bundled implementation loaded. Subsequent PDL/E8M0 messages indicate that this fallback was usable.
NVCC performance advisory	A local compilation path may benefit from a newer compiler. The run continued; this was not evidence of a failed dependency.
LBNHC cache layout	The runtime chose an internal tensor layout. It is not a statement that the cache is four-bit or that a particular capacity is available.
DSA indexer flags	use_flattening=False, supports_varlen=False, next_n=1, use_fp4_cache=False describe the selected indexer path. They do not mean that NVFP4 weights are disabled.
FlashInfer autotuning	Candidate kernel configurations were evaluated for a supported operation/shape. This does not override the logged FlashAttention sparse backend.

Compilation produces executable code. **Autotuning** chooses an implementation or launch configuration from candidates. **Warm-up** exercises paths before live requests. **CUDA graph capture** records eligible GPU operations for efficient replay. These stages can occur together but answer different questions.

The log showed an initial graph-capture phase, FlashInfer autotuning and a later graph-capture phase before readiness. Multiple captures therefore do not by themselves indicate a restart or failure. They can be part of profiling and final execution preparation.

Custom fusions included normalisation/quantisation, activation/quantisation and all-reduce/RMS operations. Fusing compatible operations can reduce intermediate traffic and launch overhead. A configuration listing enabled fusions is not a per-request trace proving that each fusion executed on every shape.

The correct operational response to a non-fatal advisory is to preserve the working baseline, identify the affected path and benchmark a controlled change. Installing every library whose name appears in a log risks replacing a working bundled implementation with an incompatible one.

Read GPU memory as a budget

The detailed memory summary was emitted for a worker. Do not interpret its values as aggregate memory for all eight GPUs, and do not silently relabel combined allocations as weights alone.

Observed worker quantity	Value
Device capacity	79.18 GiB
Free at worker startup	77.01 GiB
Target at utilisation 0.90	71.26 GiB
Consumed memory: weights plus non-Torch	55.60 GiB
Peak activation allocation	1.49 GiB
Final CUDA graph pool	0.06 GiB
Earlier available KV-cache estimate	14.18 GiB

As an approximate accounting check, $71.26 - 55.60 - 1.49 - 0.06$ is 14.11 GiB. The log also suggested a cache allocation around 13.97 GiB. Rounded measurements, allocation policy, profiling phases and reserves prevent these figures from forming an exact equality.

The initial graph-memory estimate was approximately 0.69 GiB, versus about 0.06 GiB for the final graph pool: a difference of roughly 0.63 GiB. This is an estimate-versus-measurement discrepancy, not evidence that model weights disappeared. The earlier capture report of about 0.68 GiB belonged to a different profiling stage.

`gpu-memory-utilization` defines a serving memory budget; it is not a promise to leave exactly ten per cent idle at every transient stage. Weight loading and repacking can have peaks that differ from steady inference. Other processes and non-framework allocations also matter.

Why checkpoint size is insufficient

432.90 GiB divided by eight is about 54.11 GiB, but this is only a rough average of file bytes. Runtime memory includes replication, scale representation, padding, non-quantised tensors, communication buffers and allocation overhead. The observed 55.60 GiB also included non-Torch memory, so it is not a clean weight-size comparison.

KV capacity depends on the actual layer/cache layout, dtype, block sizing, indexer state and sharing. Sparse selection limits work on selected positions; it does not necessarily reduce retained history to 2,048 tokens. Do not estimate total KV capacity by dividing all cache memory by one 576-element vector. Use the runtime's full-model capacity report and measured workload.

Compilation and CUDA graphs are distinct

The successful log contained both `CompilationMode.NONE` and `CUDAGraphMode.FULL_AND_PIECEWISE`. Graph captures completed. Those observations can coexist: disabling a high-level compilation mode does not inherently disable graph recording and replay.

The configured graph capture sizes were 1, 2 and 4. These are execution-shape budgets, not context lengths and not proof of four independent model replicas. Graphs generally reuse prepared buffers and eligible operation sequences for covered shapes, reducing CPU launch overhead.

Piecewise capture can cover selected regions where a single full graph is unsuitable. Full capture can cover a larger execution path when its constraints are satisfied. The logged mode supports both; it does not prove that every request uses the same graph path.

`VLLM_USE_BREAKABLE_CUDAGRAPH=1` was automatically enabled. This was a runtime choice in the observed build. Preserve it in the evidence record, but do not assume that it is required for all future versions or copy undocumented environment overrides indiscriminately.

What our tests established

The isolated CUDA graph test captured and replayed a small computation successfully. The tiny model then demonstrated graph-enabled integration. Finally, the target model completed graph capture and inference on eight GPUs. Each step increased coverage without making the earlier test equivalent to the later one.

`torch.compile` / Inductor / Triton passing on a synthetic operation demonstrated that this software path worked in the environment. It did not establish that the full successful model used Inductor compilation, since the final log reported mode `NONE`.

For a failure specifically during graph capture or replay, an eager-mode diagnostic can isolate graph-related constraints. If eager execution succeeds, retain the comparison and investigate capture sizes, memory and shape behaviour. Eager success does not make an unexplained graph failure acceptable for the intended production configuration.

Warm-up coverage is finite

A later request can still trigger Triton compilation, another kernel specialisation or a different graph path. Warm the expected range of prompt sizes, decode batches and concurrency. Record cold-start and warm-start latency separately. Persisted caches improve repeatability, but software and hardware changes can invalidate them.

Do not infer the exact original CLI switch from the compilation mode alone. The initial invocation was truncated. The launch template in this document leaves these controls at runtime defaults until a deliberate experiment establishes the desired mode.

What the inference log really measured

The full-model service reported one running request, no queued requests and generation rates of 88.6, 98.0, 96.6 and 96.2 tokens/s across successive intervals. KV utilisation rose from 0.6% to 2.4%. After completion, the active request count and displayed KV usage returned to zero.

Observed conclusion: generation progressed at roughly 96-98 tokens/s during several active logging intervals. The request completed with HTTP 200. This is useful operational evidence, but it is not a standardised throughput or latency benchmark.

The final interval reported 30.2 tokens/s while no request remained active. That is consistent with an interval containing only part of the request's generation period. Idle time can lower the average. Prompt throughput in periodic logs likewise must not be read as an isolated prefill-kernel rate.

Increasing KV use is consistent with a growing generated sequence. Returning to zero indicates no displayed active cache demand at that point; it does not prove the driver returned the entire preallocated pool to the operating system. Prefix-cache accounting and reusable block allocation are separate concerns.

Scheduler controls

The baseline permits 32,768 total tokens per request, at most two active sequences and a 4,096-token work budget per scheduling iteration. Chunked prefill lets a long prompt span iterations rather than requiring all prompt tokens in one batch. These are three different limits. vLLM documents the trade-off between prefill throughput and decode latency [8].

Prefix caching was enabled in the resolved configuration, but the full-model request showed a zero hit rate. That is compatible with a first or non-matching prompt. It is not evidence that the feature failed.

Measure the service you intend to operate

- Record time to first generated token, inter-token latency and complete request latency, including p50/p95/p99 across a defined workload.
- Record prompt/output lengths, concurrency, arrival rate, queue time, errors and total output throughput. Separate cold and warmed requests.
- Keep generation settings fixed. Record reasoning-token behaviour and stop reasons rather than judging only visible final-answer length.
- Test representative long prompts and concurrent requests. The supplied full-model log did not demonstrate a 32,768-token request or sustained multi-user load.

No response body was included in that full-model log. Consequently it establishes execution success, not answer correctness, reasoning quality or tool-call conformance.

Change one execution dimension at a time

Preserve the successful baseline before optimisation. For each experiment, record the change, expected effect, workload, memory on every rank, latency distribution and rollback condition. Compare warmed runs with warmed runs.

Change	Intended effect and acceptance condition
More active sequences	Improve aggregate throughput; accept only if queueing, tail latency and memory remain within the service objective.
Larger token budget	Process more prefill work per iteration; evaluate its effect on concurrent decode latency.
Longer context	Admit larger requests; verify real cache capacity, indexer behaviour and latency at the new length.
Higher memory utilisation	Increase the available cache budget; preserve headroom for transients and other allocations.
EP or alternative collectives	Change expert distribution or communication cost; revalidate routing, kernels and topology-sensitive performance.
Experimental atomic-add	Potential Marlin speed improvement for some shapes; repeat numerical and performance comparisons.

MTP speculative decoding

The inspected checkpoint included one MTP layer, but the supplied successful full-model run did not establish MTP execution. Presence of its tensors and an indexer `next_n=1` message are not proof of accepted speculative tokens.

For a supported variant, the developer's proposed experiment is to add:

```
--speculative-config '{"method":"mtp","num_speculative_tokens":1}'
```

Remove the global `--moe-backend marlin` setting for that experiment and allow backend selection per supported path. The exercised variant's MTP head was unquantised BF16; forcing one NVFP4-oriented backend globally can conflict with a heterogeneous main-model/head combination.

MTP proposes additional tokens which the main model verifies. A speed-up depends on acceptance, draft/verification cost, batch size and workload; it is not guaranteed by enabling a flag [9]. Check resolved speculative configuration, head loading, selected backends, acceptance metrics, memory and output behaviour.

Test MTP, EP, attention changes and dependency upgrades separately. For another variant, verify its prediction-head implementation and quantisation exclusions before reusing these settings.

Turn a successful launch into a service

A terminal multiplexer is appropriate for an initial manual run. A maintained service needs a supervisor, controlled shutdown, readiness handling, bounded logs and a reproducible environment. Restart policies must allow the measured loading and warm-up duration; repeatedly killing a slow but progressing startup makes diagnosis harder.

Readiness and failure isolation

Use process liveness to detect a dead server, readiness to detect completed engine startup, and a small functional request to verify useful service. A successful model listing does not exercise generation. Keep functional probes inexpensive and avoid flooding the engine while it is already overloaded.

The observed server exposed health and metrics routes as well as inference and administrative routes. Do not assume that a bearer key on inference protects every route. Restrict ingress to the intended API surface, use TLS for remote traffic, apply request-size and timeout controls, and keep internal monitoring endpoints private.

Capture the right operational evidence

Collect per-GPU memory, utilisation, temperatures and error signals; CPU utilisation and memory limits; filesystem space and I/O; request queueing, token throughput and latency. Preserve rank and process identifiers when aggregating logs. A single worker failure can surface later as a collective timeout elsewhere.

Retain the untruncated initial configuration, selected attention/MoE/prefill backends, load duration, graph and autotune summaries, cache capacity and first successful response. Store secrets separately. Do not dump the entire environment into a support ticket: it may contain credentials.

Storage and restart discipline

Weights should remain immutable during service operation. Complete a new download into a separate location, validate it, then switch deployments. Retain the previous working environment and checkpoint until rollback is no longer required. Avoid repointing a running process at a partially updated directory.

Keep caches on persistent storage when useful, but regard them as regenerable. Record their software and device context. A move to different GPUs or versions calls for new validation; a warm cache is not a substitute for compatible binaries.

For shutdown, stop admitting work, allow active requests to drain within a defined period, then terminate the server and verify the workers release their devices. Avoid leaving orphaned GPU workers before restart. Retest a clean restart as part of handover, because a service that only works once is not yet operationally reproducible.

Match the symptom to the failing stage

Symptom	Likely boundary and next action
No module named <code>vllm._C</code>	Check interpreter, package path and installed extension names. In the observed build, stable-libtorch extension names made that old import unsuitable. Then run a real operator test.
No valid attention backend with FP8 KV	Inspect the effective cache dtype. Use explicit BF16 for the validated H100 sparse path; do not rely on raw auto.
OOM during weight loading or repacking	Measure the failing rank and transient allocations. Shortening context may not solve a loading peak that precedes cache allocation.
OOM during prefill or graph capture	Review batch-token budget, concurrency, capture shapes and available headroom. Use eager execution only as a targeted comparison when capture is implicated.
NCCL hang or timeout	Find the first rank error; check visibility, topology, shared memory and the independent collective test. Do not assume the final timeout is the root cause.
DeepGEMM package absent, fallback succeeds	A bundled implementation was used. No repair is required solely because the standalone import failed.
JIT during a live request	The request encountered an unwarmed specialisation. Extend representative warm-up and separate cold/warm measurements.
Generation-config warning about <code>top_p</code>	A sampling default may not apply under that configuration. Review resolved/request sampling; this is not proof of weight corruption.
HTTP 200 with poor or repeated text	For random tiny weights this is expected. For trained weights, inspect template, tokenizer, decoding settings, output budget and model quality separately.
Blank final content or length	Inspect reasoning fields and token usage. The budget may have ended before the final answer; preserve prompt-plus-output limits.

Preserve causality

Start with the earliest failure in the relevant process, not the loudest downstream message. Change the smallest setting that tests a hypothesis. Re-run the affected acceptance check, then the full functional request. Save both failing and successful configurations so that the explanation survives beyond the shell session.

Do not treat advisory performance messages as installation failures. Conversely, a server that eventually returns one response has not automatically passed sustained load, correctness, long-context or tool-use acceptance.

The acceptance record to retain

The most useful deliverable from deployment is a repeatable record of what ran, why it was selected and what remains untested. Preserve the exact private model identifier in your internal manifest even though this public guide deliberately omits it.

Evidence group	Minimum record
Host	GPU models/count/topology, driver, operating-system image or digest, CPU and RAM limits, shared memory, filesystem and storage capacity.
Software	Python executable/version, resolved packages, package origins, CUDA runtime, compiler versions when used, vLLM help/version output.
Checkpoint	Approved immutable identifier, configuration, tokenizer/template, shard manifest and integrity verification policy.
Execution	Complete launch script, non-secret overrides, resolved runtime configuration, rank layout and backend selections.
Validation	Scripts, shapes, seeds, reference calculations, numerical errors, collective result and first functional request/response.
Operations	Cold/warm startup times, per-rank memory, metrics, readiness policy, ingress configuration, shutdown and rollback procedure.

What is already supported by this case

The observed stack executed compiler/graph tests, generic attention, NVFP4 Marlin GEMM, routed fused MoE and BF16 sparse MLA on H100. A tiny related architecture completed serving integration. The full target checkpoint then loaded with TP=8 and served a chat-completions request.

The strongest configuration finding was the need to test the effective KV dtype: a generic FP8 success did not imply sparse-MLA compatibility. Explicit BF16 matched the actual selected backend. The strongest operational finding was that readiness followed several distinct stages beyond shard loading.

What still requires workload-specific acceptance

Sustained concurrency, maximum intended context, answer quality, structured tool calls, complete reasoning output, MTP benefit, cold-start variance, restart reliability and performance on a different host were not established by the supplied evidence. Treat these as clearly bounded next steps rather than silently inheriting confidence from a different test.

A later upgrade should rerun the smallest relevant checks first, then a full functional and representative-load test. Keep the working release available until the new one meets the same criteria. The stable object is the acceptance procedure, not a permanently correct set of flags.

References and provenance

Case evidence. Numerical results, configuration observations and timings in this guide come from the operator-supplied validation outputs and the full-model vLLM log dated 19 September 2026. Earlier tiny-model outputs are identified separately. The full-model log is partially truncated at its beginning; the complete launch command and response body were not available.

Documentation. The following primary sources were consulted on 19 September 2026. Numbered references support the associated sections; the detailed numerical case findings are local observations, not claims taken from these web pages.

1. [vLLM 0.29.0 - Architecture Overview](#) — process responsibilities and execution structure.
2. [NVIDIA - CUDA Minor Version Compatibility](#) — driver compatibility for CUDA runtime families.
3. [vLLM 0.29.0 - GPU Installation](#) — isolated environments, wheels and binary compatibility.
4. [FlashInfer - Installation](#) — Python, cubin and CUDA-specific JIT-cache packages.
5. [Hugging Face Hub - Command Line Interface](#) — authenticated acquisition and local-directory downloads.
6. [vLLM 0.29.0 - Cache Configuration](#) — explicit cache dtype and cache configuration surface.
7. [vLLM 0.29.0 - OpenAI-Compatible Server](#) — serving endpoints, requests and model naming.
8. [vLLM 0.29.0 - Optimisation and Tuning](#) — chunked prefill and scheduler trade-offs.
9. [vLLM 0.29.0 - Speculative Decoding](#) — speculative execution and workload-dependent performance.

Interpretation policy. The document distinguishes checkpoint bytes from runtime memory, generic kernel support from selected model support, and transport success from answer quality. Recommendations are scoped to a fresh deployment based on the observed stack. No claim is made that all GLM-5.3 derivatives have identical precision, dimensions or execution requirements.